

Simulación de Entrevista Técnica — POS Microservices System

Guía de estudio en formato entrevista · sistema POS cloud-native (Spring Boot · Kafka · SAGA · Angular)

Guía de estudio en formato entrevista sobre este proyecto (sistema POS cloud-native con microservicios Spring Boot, Kafka, SAGA y Angular). Cada sección incluye la **pregunta del entrevistador**, una **respuesta modelo** y, cuando aplica, **preguntas de seguimiento** (follow-ups) que suelen aparecer.

Formato sugerido: 60–75 min. Bloques: arquitectura (15'), SAGA/mensajería (20'), datos (10'), seguridad (10'), frontend (10'), DevOps/testing (10'), diseño abierto (10').

0. Warm-up — "Cuéntame del proyecto"

P En 2–3 minutos, describe el sistema y tu rol.

R Es un sistema de punto de venta (POS) construido con arquitectura de microservicios. Hay seis servicios backend en Java 21 / Spring Boot 3.3:

- **api-gateway** (8080): único punto de entrada, Spring Cloud Gateway + Eureka.
- **auth-service** (8084): autenticación JWT con PostgreSQL.
- **stock-service** (8081): inventario, MongoDB + Kafka.
- **venta-service** (8082): orquestador de ventas, implementa el patrón SAGA.
- **despacho-service** (8083): despachos/envíos, MongoDB + Kafka.
- **eureka-server** (8761): service discovery.

Tres frontends Angular 18: `pos-frontend` (4300), `ventas-mantenedor` (4200) y `users-mantenedor` (4400). La infraestructura es Kafka, MongoDB, PostgreSQL y Redis, con Jenkins y SonarQube para CI/CD y calidad. Todo se orquesta con Docker Compose (y manifiestos de Kubernetes en `k8s/`). El caso de negocio central es una **venta como transacción distribuida**: crear la orden, reservar stock y solicitar despacho, con compensación automática si algo falla.

1. Arquitectura

P1 ¿Por qué microservicios y no un monolito para un POS?

R Se justifica por: (1) **límites de dominio claros** — inventario, ventas y despacho evolucionan y escalan por separado; (2) **escalado independiente** — el stock y las ventas tienen perfiles de carga distintos; (3) **aislamiento de fallos** — si despacho cae, la reserva de stock y la captura de la venta siguen. El costo es complejidad operativa (mensajería, consistencia eventual, observabilidad), que aquí se acepta como ejercicio de diseño. Para un POS pequeño, un monolito modular sería defendible; la decisión aquí es explícitamente didáctica y orientada a demostrar SAGA.

P2 ¿Qué rol cumple el API Gateway y por qué Eureka?

R El gateway (Spring Cloud Gateway) es el único punto de entrada: enruta a los servicios, centraliza CORS, y puede aplicar rate limiting con Redis. Eureka da **service discovery** dinámico: los servicios se registran y el gateway los resuelve por nombre lógico en vez de IP/puerto fijos, lo que permite escalar réplicas sin reconfigurar. En Kubernetes este rol lo podría asumir el DNS del cluster; Eureka se mantiene por portabilidad.

FOLLOW-UP ¿Y si Eureka se cae? — Los clientes cachean el registro, así que hay degradación elegante por un tiempo; aun así es un punto a replicar en producción (varias instancias en zona/región).

P3 ¿Comunicación síncrona vs asíncrona en este sistema?

R Las consultas cliente → servicio son **síncronas** (REST vía gateway). El flujo de venta entre servicios es **asíncrono** vía Kafka (eventos). Esto desacopla al orquestador de los participantes y da tolerancia a picos y a caídas temporales de un consumidor (los mensajes quedan en el topic).

2. Patrón SAGA y mensajería (núcleo del proyecto)

P4 Explica el flujo SAGA de una venta.

R `venta-service` es el **orquestador**. El flujo:

1. `POST /api/ventas` crea la orden en estado **PENDING** y produce a `stock-reserve-topic`.
2. `stock-service` (participante) consume, valida `quantity - reservedQuantity >= solicitado`, reserva y responde en `stock-reserve-response-topic`.
3. Si `success=true` → orden pasa a **STOCK_RESERVED** y venta produce a `despacho-request-topic`. Si `success=false` → **STOCK_FAILED** (terminal).
4. `despacho-service` consume, genera un tracking `TRK-XXXXXXX`, crea el despacho en **PROCESSING** y responde en `despacho-response-topic`.
5. Si `success=true` → orden **COMPLETED**. Si `success=false` → **DISPATCH_FAILED** y venta produce a `stock-compensate-topic` para **liberar** el stock reservado (compensación).

Es una **SAGA por orquestación**: un coordinador central dirige los pasos y dispara las compensaciones, a diferencia de la coreografía donde cada servicio reacciona a eventos sin coordinador.

P5 ¿Orquestación vs coreografía? ¿Por qué orquestación aquí?

R Orquestación centraliza la lógica del flujo y las compensaciones en un solo lugar (venta), lo que hace el flujo **explícito, testeable y fácil de observar** — clave con solo tres pasos y compensación no trivial. Coreografía escala mejor a muchos servicios y evita un coordinador central, pero dispersa la lógica y complica el rastreo. Con este tamaño, orquestación es la elección correcta.

P6 ¿Cómo se implementa la **compensación** y por qué no un rollback ACID?

R No hay transacción distribuida ACID (no hay 2PC entre MongoDB de distintos servicios). En su lugar, cada paso tiene una **acción compensatoria semántica**: la reserva de stock se compensa liberando la cantidad (`stock-compensate-topic`). Es **consistencia eventual**: durante la SAGA el sistema puede estar temporalmente inconsistente (stock reservado pero venta no completada), y las compensaciones lo reconcilian. Se evita 2PC porque bloquea recursos, acopla servicios y escala mal.

FOLLOW-UP ¿Y si la compensación falla? — Hay que hacerla **idempotente y reintentable**; en producción se añade DLQ (dead-letter topic), reintentos con backoff y alertas para intervención manual como último recurso.

P7 Idempotencia y entrega "at-least-once": ¿qué problemas y cómo los mitigas?

R Kafka garantiza *al menos una vez* por defecto, así que un consumidor puede procesar el mismo evento dos veces (p. ej. tras un rebalanceo). Riesgos: reservar stock dos veces, crear dos despachos. Mitigaciones:

- **Claves de idempotencia** por `orderId` (la regla "no dos despachos para la misma orden" del spec de despacho apunta a esto).
- Operaciones **idempotentes** (upsert por `orderId`; verificar estado actual antes de aplicar).
- **Deduplicación** por evento procesado.
- Diseñar las compensaciones para ser seguras si se repiten.

FOLLOW-UP ¿Orden de mensajes? — Kafka ordena **por partición**. Particionar por `orderId` garantiza orden dentro de una misma orden, que es lo que importa aquí.

P8 ¿Qué topics existen y quién produce/consume?

R

Topic	Produce	Consume
<code>stock-reserve-topic</code>	venta	stock
<code>stock-reserve-response-topic</code>	stock	venta
<code>despacho-request-topic</code>	venta	despacho
<code>despacho-response-topic</code>	despacho	venta
<code>stock-compensate-topic</code>	venta	stock

Cada servicio tiene su **consumer group** (`venta-group` , `stock-group` , `despacho-group`), lo que permite escalar consumidores dentro de un grupo.

3. Datos y persistencia

P9 ¿Por qué MongoDB para stock/venta/despacho y PostgreSQL para auth?

R **Database-per-service:** cada servicio es dueño de su base y nadie accede a la de otro directamente. MongoDB (documental) encaja con agregados autocontenidos como `Order`, `Product` y `Dispatch`, con esquema flexible y escritura simple. PostgreSQL (relacional) para **auth** porque usuarios, roles y credenciales se benefician de integridad referencial y consultas relacionales. Es "poliglote persistence": elegir el motor por caso de uso.

FOLLOW-UP ¿Cómo consultas datos que cruzan servicios (p. ej. venta + producto + despacho)? — No con JOINS entre bases. Opciones: composición vía API (el gateway/BFF agrega), o vistas materializadas por eventos (CQRS/read model). Aquí el frontend compone llamando a varios endpoints.

P10 Modela el estado de una orden. ¿Qué estados son terminales?

R `OrderStatus`: `PENDING` → `STOCK_RESERVED` → `(DISPATCHED)` → `COMPLETED` en el camino feliz. Terminales de fallo: **STOCK_FAILED** (no hubo stock) y **DISPATCH_FAILED** (falló el despacho, ya compensado). `CANCELLED` para cancelación. Modelarlo como máquina de estados evita transiciones inválidas (regla: "solo órdenes con stock reservado pueden solicitar despacho").

4. Seguridad

P11 ¿Cómo funciona la autenticación con JWT aquí?

R `auth-service` valida credenciales contra PostgreSQL y emite un **JWT** firmado (HS256) con expiración (`JWT_EXPIRATION`). El cliente lo envía en `Authorization: Bearer`. El gateway/servicios validan la firma con `JWT_SECRET`. JWT es **stateless**: no hay sesión en servidor, lo que escala bien horizontalmente.

FOLLOW-UP 1 ¿Dónde guardas el token en el frontend y qué riesgos hay? — Típicamente en memoria o `localStorage`. `localStorage` es vulnerable a XSS; cookies `HttpOnly` + `SameSite` mitigan XSS pero requieren cuidado con CSRF. Se elige según el modelo de amenazas.

FOLLOW-UP 2 ¿Cómo revocas un JWT antes de que expire? — JWT puro no se revoca; se usan listas de revocación/denylist (Redis), rotación de claves o tokens de vida corta + refresh tokens.

P12 (Trabajo reciente en este repo) Se pidió **dobles validación de contraseña** en el alta de usuario (users -mantenedor). ¿Cómo lo resolviste y qué falta para que sea seguro?

R En el formulario reactivo de Angular añadí un segundo campo `confirmPassword` y un **validador a nivel de grupo** (`passwordsMatch`) que compara ambos y marca el error `passwordMismatch` , deshabilitando el submit y mostrando "Las contraseñas no coinciden". En **crear** ambos son requeridos; en **editar** son opcionales (vacío = no cambiar) y solo se valida coincidencia si el usuario escribe algo.

Importante: esto es **validación de UX en el cliente**. La confirmación no se envía al backend (solo `password`). Para seguridad real hace falta: validación y reglas de fortaleza en `auth-service` , **hashing** con bcrypt/argon2 (nunca texto plano), y política de contraseñas server-side. El cliente nunca es la frontera de seguridad.

P13 ¿Rate limiting? ¿Para qué está Redis?

R Redis actúa como store para **rate limiting** en el gateway (y como caché). Limitar peticiones por cliente/IP protege de abuso y fuerza bruta en login. Spring Cloud Gateway tiene un `RequestRateLimiter` basado en Redis (token bucket).

5. Frontend (Angular)

P14 ¿Cómo se sirven los frontends en contenedor y qué implicación tiene?

R Build multi-stage: etapa Node hace `ng build` (Angular 18, componentes *standalone*), y una etapa **Nginx** sirve el `dist/` estático. Implicación clave: el bundle queda **congelado en la imagen**. Editar el código fuente no cambia lo que ve el usuario hasta **reconstruir la imagen y recrear el contenedor** — justo un caso que viví: un cambio no aparecía porque el contenedor servía el build anterior; se resolvió con:

```
docker compose up -d --build --force-recreate <servicio>
```

FOLLOW-UP

¿Cómo evitas caché del navegador tras un deploy? — Angular genera nombres de bundle con hash (`main-XXXX.js`), lo que invalida caché automáticamente; un hard refresh (Ctrl+F5) fuerza recarga del `index.html` .

P15

¿Formularios reactivos vs template-driven? ¿Por qué reactivos aquí?

R

Reactivos (`ReactiveFormsModule` , `FormBuilder`) porque permiten **validadores compuestos y a nivel de grupo** (como el de contraseñas), lógica testeable, y control programático del estado del form. Template-driven sirve para formularios triviales; para validaciones cruzadas, reactivos.

6. DevOps, testing y tooling

P16

Describe el pipeline CI/CD.

R

Jenkins con etapas: Checkout → Build (Gradle) → Test (unit + integración) → SonarQube (análisis + cobertura JaCoCo) → Docker (build/push) → Deploy (Kubernetes). El *quality gate* exige, según el README, ~80% de cobertura y cero issues críticos/blocker.

P17

¿Cómo se testea una SAGA de forma fiable?

R

Tres niveles: (1) **unitarios** de la lógica de transición de estados y compensación (sin infra); (2) **integración con Testcontainers** levantando MongoDB/PostgreSQL/Kafka reales para verificar producción/consumo de eventos end-to-end de cada servicio; (3) **E2E** (Playwright, carpeta `e2e/`) y **carga** con Gatling (`stress-test/`). Testcontainers es clave porque los mocks de Kafka no capturan rebalanceos, offsets ni serialización real.

P18 El README menciona optimizaciones de arranque. ¿Cuáles y por qué?

R (1) **Perfiles de Compose**: tooling pesado (Jenkins, SonarQube) va tras perfiles y no arranca por defecto → stack más liviano. (2) **Límites de CPU** de 2 CPU/768M por JVM porque el arranque de Spring Boot es CPU-bound (JIT, class scanning); bajarlo a 0.5 CPU casi duplica el tiempo. (3) **Cachés BuildKit compartidas** de Gradle y npm entre servicios → dependencias se descargan una vez. (4) `-Djava.security.egd=file:/dev/./urandom` para no bloquear por entropía. (5) **Arranque por fases** (infra → backend → frontend) esperando health checks. Hay scripts en `services-script/` para esto.

P19 (Trabajo reciente) Construíste una **GUI de escritorio** para gestionar el stack. Cuéntala y justifica decisiones.

R Es `services-script/stack_gui.py`, una GUI **GTK3 nativa** (PyGObject) con tres pestañas: **Control** (botones subir/bajar por fase, stack completo, Jenkins, Sonar; cada tarjeta muestra estado Arriba/Parcial/Abajo por servicio; consola con **traza en vivo** por streaming línea a línea con `Popen`), **Logs** (docker logs por contenedor) y **Recursos** (CPU/RAM por contenedor con alertas). Decisiones: (a) reescribí una versión web previa a **escritorio** por requisito; (b) elegí **GTK3** porque PyGObject ya estaba disponible sin `sudo` ni `pip`, fiel al espíritu "solo stdlib"; (c) las operaciones bloqueantes corren en **hilos** y actualizan la UI con `GLib.idle_add`, con timers `GLib.timeout_add`; (d) la ventana se **ajusta al área de trabajo del monitor** y usa un `Gtk.Paned` para que la consola nunca quede fuera de pantalla; (e) las acciones están en un **whitelist** que solo ejecuta los scripts `.sh` del repo (no comandos arbitrarios). Para verificarla en un entorno headless (sin X ni `sudo` para `xvfb`) usé el backend **Broadway** de GTK, que renderiza en memoria sin servidor X.

7. Diseño abierto / escenarios

P20 El `despacho-service` cae por 30 minutos. ¿Qué pasa con las ventas?

R Las órdenes quedan en **STOCK_RESERVED** con su evento en `despacho-request-topic` (Kafka lo retiene). Al volver despacho, consume el backlog y responde; la SAGA continúa. Riesgo: stock reservado retenido mientras tanto. Mitigaciones: timeouts en la SAGA que compensen si despacho no responde en X, y monitoreo del lag del consumer group.

P21 ¿Cómo agregarías **observabilidad** de una venta que atraviesa 3 servicios y Kafka?

R **Tracing distribuido** (OpenTelemetry/Zipkin) propagando un `traceId` / `correlationId` desde el gateway y a través de las cabeceras de los eventos Kafka, para reconstruir la línea de tiempo completa. Más **métricas** (Actuator/Micrometer → Prometheus: lag de Kafka, tasa de compensaciones, duración de SAGA) y **logs estructurados** correlacionados por `orderId`.

P22 Llega Black Friday. ¿Qué escalas primero y cómo?

R Perfilar antes de escalar. Probables cuellos: `stock-service` (mucha contención de reservas) y `venta-service`. Escalado horizontal de esos consumidores (más particiones por `orderId` + más instancias en el consumer group), réplicas del gateway, y cuidar el hotspot de reservas (posible optimistic locking en Mongo). Kafka absorbe picos como buffer natural.

8. Preguntas de cierre (para el candidato)

- ¿Qué **rediseñarías**? — Añadir DLQ + reintentos con backoff a los consumidores; timeouts/compensación por vencimiento en la SAGA; tracing distribuido; y mover la validación de contraseña también al backend con hashing fuerte.
- ¿Mayor **deuda técnica**? — Consistencia eventual sin timeouts explícitos y compensaciones no garantizadas como idempotentes; falta de observabilidad transversal.
- ¿De qué estás **orgulloso**? — Del flujo SAGA limpio por orquestación con compensación explícita y del tooling de operación (GUI + scripts por fases) que hace el stack manejable localmente.

Apéndice — Chuleta rápida

Tema	Punto clave
Arquitectura	6 microservicios Spring Boot + 3 frontends Angular + gateway/Eureka
SAGA	Orquestada por venta-service; compensación vía <code>stock-compensate-topic</code>
Consistencia	Eventual, no 2PC; compensaciones semánticas idempotentes
Kafka	at-least-once → idempotencia por <code>orderId</code> ; orden por partición
Datos	DB-per-service; Mongo (dominio) + Postgres (auth)
Seguridad	JWT stateless HS256; rate limiting con Redis; validar+hashear en server
Frontend	Angular standalone, build estático servido por Nginx (rebuild para desplegar)
CI/CD	Jenkins: build → test → sonar → docker → k8s; Testcontainers para integración
Arranque	Perfiles Compose, CPU 2/768M por JVM, cachés BuildKit, fases con health checks